Universidad Veracruzana



Facultad de Estadística e Informática

Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Peticiones sobre Protocolo HTTP

Experiencia Educativa: Matemáticas para Ciberseguridad

Académico: José Pablo Rueda Holguín

Alumno: Javier Patiño González

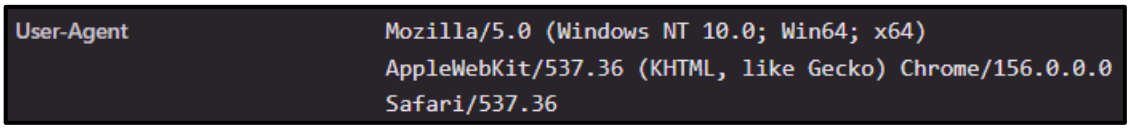
- ## Actividad Parte 1

Se requiere analizar la respuesta que ofrecen los servicios web a través de sus cabeceras, revisar los elementos que las componen y el como pueden variar a través de los diferentes dominios web que existen.

Se analizará las cabeceras y respuestas de 6 dominios web distintos, se obtendrá la línea de la solicitud (método, ruta, versión), la línea de estado (versión, código, frase), servidor, Content-Type y el User-Agent con el que se presenta el navegador, acompañado de capturas de pantalla.

- User-Agent

En todos los casos, el User-Agent es el mismo, pues representa al navegador que uso para acceder a los recursos, es el siguiente:

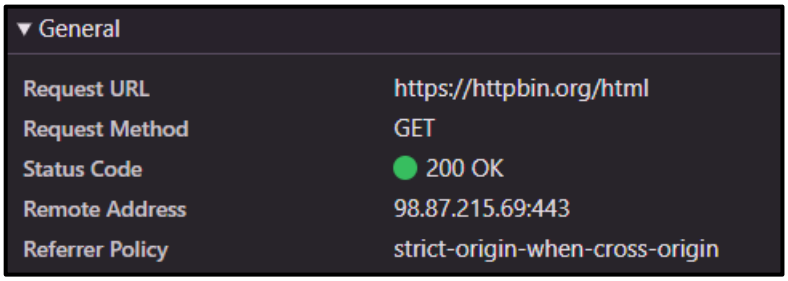


- ## Lista de Dominios

- https://httpbin.org/html

Cabeceras principales

- URL buscado, enlace del servidor
- Tipo de petición, GET solicita información
- Código de estado, 200 OK implica que fue exitosa la petición
- Dirección y puerto del servidor

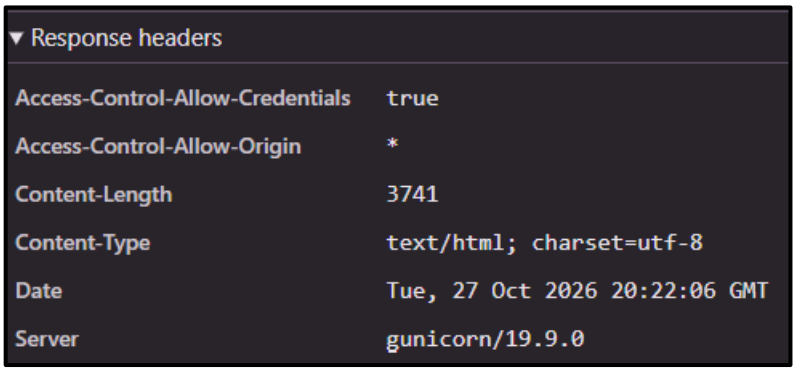


Cabeceras con más respuestas, alguna información que incluye es

- Tamaño del contenido

- Fecha de la petición

- Sistema operativo y versión del servidor

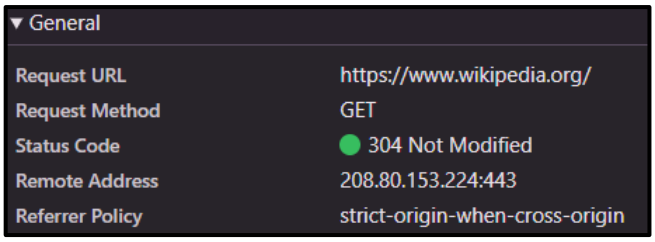


Petición en crudo desde la terminal, gran parte de la información es similar.

```
C:\Users\JavoPC>curl -I https://httpbin.org/html
HTTP/1.1 200 OK
Date: Tue, 27 Oct 2026 21:02:09 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3741
Connection: keep-alive
Server: gunicorn/19.9.0
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

Los siguientes ejemplos cambian ligeramente pero siguen la misma estructura general

- https://www.wikipedia.org/



▼ General

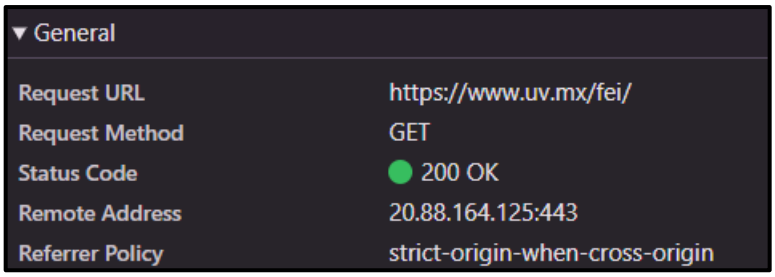
Request URL	https://www.wikipedia.org/
Request Method	GET
Status Code	304 Not Modified
Remote Address	208.80.153.224:443
Referrer Policy	strict-origin-when-cross-origin

▼ Response headers

Accept-Ranges	bytes
Age	20855
Cache-Control	s-maxage=86400, must-revalidate, max-age=3600
Content-Encoding	gzip
Content-Type	text/html
Date	Tue, 27 Oct 2026 15:01:30 GMT
Etag	W/"1de35-65e335ecb78c0"
Last-Modified	Mon, 19 Oct 2026 15:43:07 GMT
Nel	{ "report_to": "wm_nel", "max_age": 604800, "failure_fraction": 0.05, "success_fraction": 0.0}
Report-To	{ "group": "wm_nel", "max_age": 604800, "endpoints": [{ "url": "https://intake-logging.wikimedia.org/v1/events?stream=w3c.reportingapi.network_error&schema_uri=/w3c/reportingapi/network_error/1.0.0" }] }
Server	ATS/9.2.15

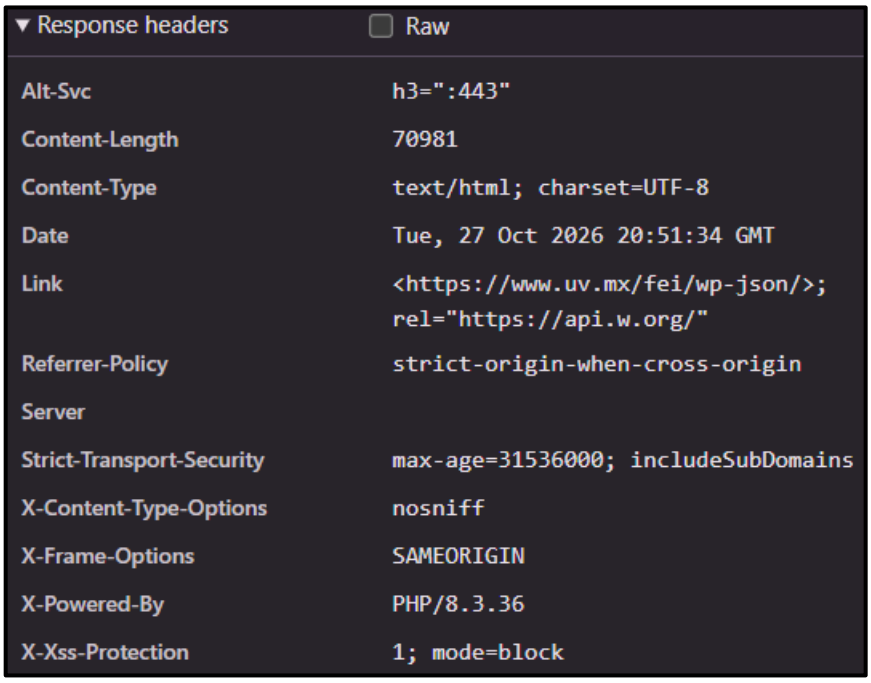
```
C:\Users\JavoPC>curl -I https://www.wikipedia.org/
HTTP/1.1 200 OK
date: Tue, 27 Oct 2026 15:01:30 GMT
cache-control: s-maxage=86400, must-revalidate, max-age=3600
server: ATS/9.2.15
etag: W/"1de35-65e335ecb78c0"
last-modified: Mon, 19 Oct 2026 15:43:07 GMT
content-type: text/html
age: 21700
accept-ranges: bytes
x-cache: cp2046 miss, cp2046 hit/881941
x-cache-status: hit-front
strict-transport-security: max-age=106384710; includeSubDomains
report-to: { "group": "wm_nel", "max_age": 604800, "endpoints":
s?stream=w3c.reportingapi.network_error&schema_uri=/w3c/reporti
nel: { "report_to": "wm_nel", "max_age": 604800, "failure_fract
set-cookie: WMF-Last-Access=27-Oct-2026;Path=/;HttpOnly;secure;
set-cookie: WMF-Last-Access-Global=27-Oct-2026;Path=/;Domain=.w
00:00 GMT
x-client-ip: 148.226.28.247
set-cookie: GeoIP=MX:VER:Xalapa:19.54:-96.89:v4; Path=/; secure
set-cookie: NetworkProbeLimit=0.001;Path=/;Secure;SameSite=None
set-cookie: WMF-Uniq=by8j55lIaf80ISa_bquUGwQGAAAAAFvdfNGp-pGv0I
nly;secure;SameSite=None;Expires=Wed, 27 Oct 2027 00:00:00 GMT
content-length: 122421
```

- https://www.uv.mx/fei/



▼ General

Request URL	https://www.uv.mx/fei/
Request Method	GET
Status Code	200 OK
Remote Address	20.88.164.125:443
Referrer Policy	strict-origin-when-cross-origin



▼ Response headers ☐ Raw

Alt-Svc	h3=":443"
Content-Length	70981
Content-Type	text/html; charset=UTF-8
Date	Tue, 27 Oct 2026 20:51:34 GMT
Link	<https://www.uv.mx/fei/wp-json/>; rel="https://api.w.org/"
Referrer-Policy	strict-origin-when-cross-origin
Server	
Strict-Transport-Security	max-age=31536000; includeSubDomains
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN
X-Powered-By	PHP/8.3.36
X-Xss-Protection	1; mode=block

```
C:\Users\JavoPC>curl -I https://www.uv.mx/fei/
HTTP/1.1 200 OK
Content-Length: 0
Content-Type: text/html; charset=UTF-8
Server:
X-Powered-By: PHP/8.3.36
Link: <https://www.uv.mx/fei/wp-json/>; rel="https://api.w.org/"
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Strict-Transport-Security: max-age=31536000; includeSubDomains
alt-svc: h3=":443"
Date: Tue, 27 Oct 2026 21:04:38 GMT
Set-Cookie: cookiesession1=678A3E208AEDAA3F5FC2EFF71A49444E;Expires=Wed, 27 Oct 2027 21:04:39 GMT;Path=/
```

- https://filezilla-project.org/

▼ General

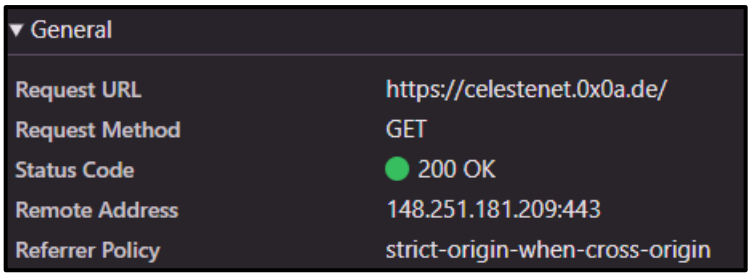
Request URL	https://filezilla-project.org/
Request Method	GET
Status Code	200 OK
Remote Address	49.12.121.47:443
Referrer Policy	strict-origin-when-cross-origin

▼ Response headers

Content-Encoding	gzip
Content-Length	16581
Content-Security-Policy	default-src 'self' *.filezilla-project.org; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline'; img-src * blob:
Content-Type	text/html; charset=UTF-8
Date	Tue, 27 Oct 2026 20:56:04 GMT
Server	Apache

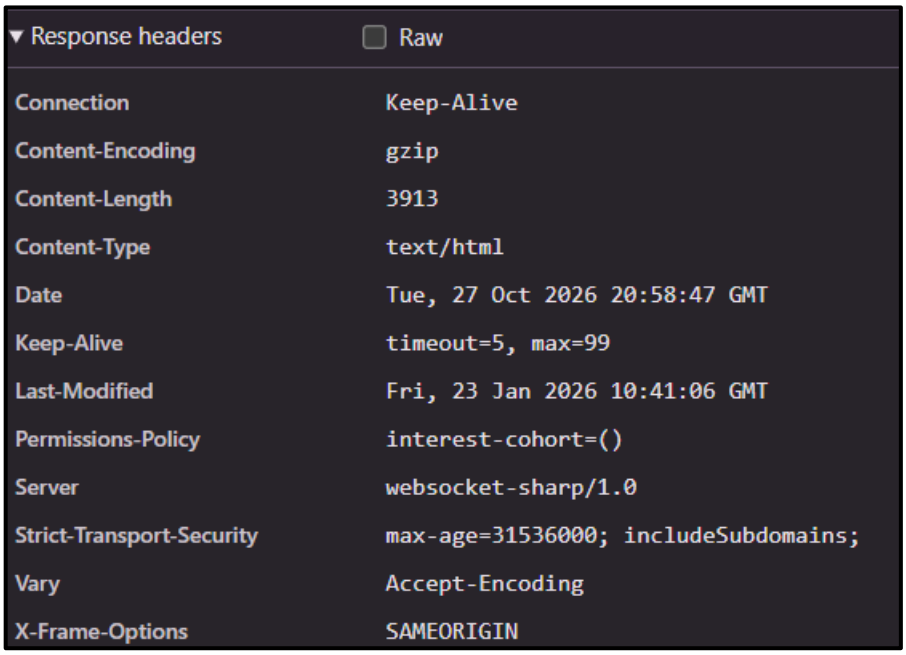
```
C:\Users\JavoPC>curl -I https://filezilla-project.org/
HTTP/1.1 200 OK
Date: Tue, 27 Oct 2026 21:05:38 GMT
Server: Apache
Content-Security-Policy: default-src 'self' *.filezilla-project.org; script-src
yle-src 'self' 'unsafe-inline'; img-src * blob:
Strict-Transport-Security: max-age=315360000; includeSubDomains; preload
Upgrade: h2
Connection: Upgrade
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
X-Frame-Options: deny
Content-Type: text/html; charset=UTF-8
```

- https://celestenet.0x0a.de/



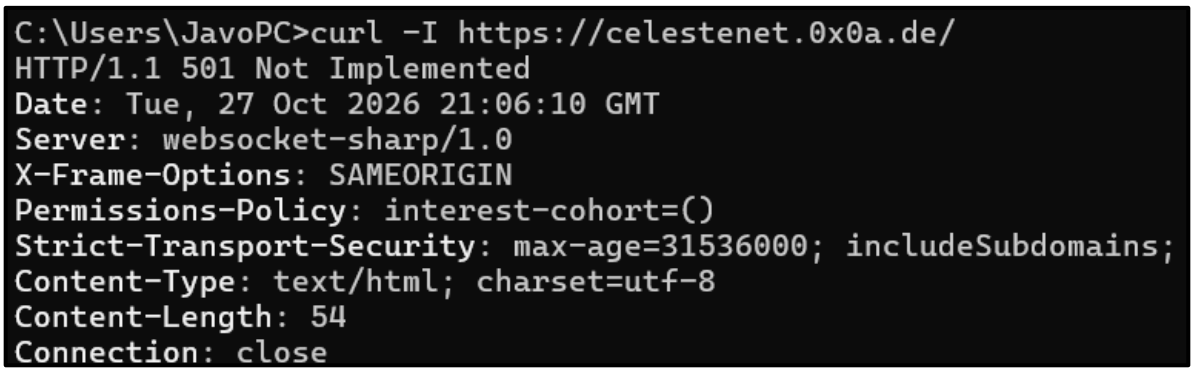
▼ General

Request URL	https://celestenet.0x0a.de/
Request Method	GET
Status Code	200 OK
Remote Address	148.251.181.209:443
Referrer Policy	strict-origin-when-cross-origin



▼ Response headers ☐ Raw

Connection	Keep-Alive
Content-Encoding	gzip
Content-Length	3913
Content-Type	text/html
Date	Tue, 27 Oct 2026 20:58:47 GMT
Keep-Alive	timeout=5, max=99
Last-Modified	Fri, 23 Jan 2026 10:41:06 GMT
Permissions-Policy	interest-cohort=()
Server	websocket-sharp/1.0
Strict-Transport-Security	max-age=31536000; includeSubdomains;
Vary	Accept-Encoding
X-Frame-Options	SAMEORIGIN



```
C:\Users\JavoPC>curl -I https://celestenet.0x0a.de/
HTTP/1.1 501 Not Implemented
Date: Tue, 27 Oct 2026 21:06:10 GMT
Server: websocket-sharp/1.0
X-Frame-Options: SAMEORIGIN
Permissions-Policy: interest-cohort=()
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Type: text/html; charset=utf-8
Content-Length: 54
Connection: close
```

- https://www.youtube.com/

▼ General

Request URL	https://www.youtube.com/
Request Method	GET
Status Code	200 OK (from service worker)
Referrer Policy	strict-origin-when-cross-origin

Content-Type	text/html; charset=utf-8
Cross-Origin-Opener-Policy	same-origin-allow-popups; report-to="youtube_main"
Date	Tue, 27 Oct 2026 21:09:03 GMT
Document-Policy	include-js-call-stacks-in-crash-reports
Expires	Mon, 01 Jan 1990 00:00:00 GMT

Server	ESF
Strict-Transport-Security	max-age=31536000
Vary	Sec-CH-UA-Arch, Sec-CH-UA-Bitness, Sec-CH-UA-Full-Version, Sec-CH-UA-Full-Version-List, Sec-CH-UA-Model, Sec-CH-UA-WoW64, Sec-CH-UA-Form-Factors, Sec-CH-UA-Platform, Sec-CH-UA-Platform-Version, Sec-CH-Viewport-Width, Sec-CH-DPR, Device-Memory
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN

- Cuestionario Parte 1

¿Por qué el encabezado Host es obligatorio en HTTP/1.1 si la conexión ya sabe a qué IP llegó?

- Porque pueden surgir varias peticiones diferentes que requiera de muchas conexiones, si solo respondiera a la IP que llego, si se conserva el encabezado Host, puede responder todas esas peticiones a ese destino sin necesidad de abrir muchas conexiones.

Si HTTP es texto plano y legible, ¿qué implica eso para las contraseñas que viajan en un formulario? (anticipo de la clase de HTTPS)

- Esto significa que cualquier persona que este dentro de la ruta de la conexión podría interceptar esta información y tener acceso ilícito a estas contraseñas.

¿Qué tienen en común el protocolo apodo>mensaje de la Unidad I, el XML-RPC de la semana pasada y HTTP?

- Que se encuentran formateados de tal manera que si ambos extremos pueden entender el mismo formato, ambos pueden comunicar información de manera consistente, dando a lugar al concepto de los protocolos de red, la estandarización de reglas a la hora de transmitir información en red.

- ## Actividad Parte 2

Ya analizadas las cabeceras que arrojaban los dominios web al tratar de acceder a ellos, ahora convendría familiarizarnos con todos los tipos de códigos que existen y bajo que circunstancias pueden salir, por lo que esta vez, se deberá hacer peticiones especificas en una consola haciendo uso de herramientas como **curl** y **netcat.**

- ## Prueba de Peticiones

Ejercicio 1 – Un 200 a mano

- Con curl -v pidan https://httpbin.org/get. Identifiquen en la salida: la línea de solicitud que ustedes generaron, el User-Agent de curl y la línea de estado 200.

```
C:\Users\JavoPC>curl -v https://httpbin.org/get
* Host httpbin.org:443 was resolved.
* IPv6: (none)
* IPv4: 3.220.166.81, 34.201.225.204, 35.170.125.171, 52.6.78.96, 52.44.60.253,
 98.87.215.69, 184.194.160.148, 3.211.106.232
*   Trying 3.220.166.81:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to httpbin.org (3.220.166.81 port 443) from 10.30.22.7
6 port 26122
* using HTTP/1.x
> GET /get HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/8.21.0
> Accept: */*
```

- Se puede percibir que la petición es “GET /get HTTP/1.1”
- El User-Agent es “curl/8.21.0”

```
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 27 Oct 2026 21:51:42 GMT
< Content-Type: application/json
< Content-Length: 256
< Connection: keep-alive
< Server: gunicorn/19.9.0
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
<
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Host": "httpbin.org",
    "User-Agent": "curl/8.21.0",
    "X-Amzn-Trace-Id": "Root=1-6ae11cee-74a9bf4976d93fa108cdf040"
  },
  "origin": "148.226.28.247",
  "url": "https://httpbin.org/get"
}
* Connection #0 to host httpbin.org:443 left intact
```

- Y la respuesta de la petición es “HTTP/1.1 200 OK”

Ejercicio 2 – Un 404 provocado

- Pidan una ruta que no existe: curl -v https://httpbin.org/noexiste. Registren la línea de estado y expliquen por qué es un error 4xx y no 5xx.

```
C:\Users\JavoPC>curl -v https://httpbin.org/noexiste
* Host httpbin.org:443 was resolved.
* IPv6: (none)
* IPv4: 44.206.195.5, 44.210.74.85, 52.44.60.253, 98.87.215.69, 184.194.160.148, 3.220.166.81, 34
.201.225.204, 35.170.125.171
*   Trying 44.206.195.5:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to httpbin.org (44.206.195.5 port 443) from 10.30.22.76 port 57619
* using HTTP/1.x
> GET /noexiste HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404 NOT FOUND
< Date: Tue, 27 Oct 2026 21:18:35 GMT
< Content-Type: text/html
< Content-Length: 233
< Connection: keep-alive
< Server: gunicorn/19.9.0
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
<
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server.  If you entered the URL manually please check y
our spelling and try again.</p>
* Connection #0 to host httpbin.org:443 left intact
```

- La línea de estado es “HTTP/1.1 404 NOT FOUND”
- El error es de la familia 4xx y no 5xx porque se trata de un error de cliente, al tratar de acceder a un recurso inexistente, no puede ser culpa del servidor.

Ejercicio 3 – Un 500 bajo perdido

- Pidan https://httpbin.org/status/500: el servidor simula una falla interna. Registren la respuesta y expliquen la diferencia de responsabilidad frente al 404.

```
C:\Users\JavoPC>curl -v https://httpbin.org/status/500
* Host httpbin.org:443 was resolved.
* IPv6: (none)
* IPv4: 3.137.48.32, 16.58.42.220, 18.227.178.96
*   Trying 3.137.48.32:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to httpbin.org (3.137.48.32 port 443) from 10.30.22.76 port
13376
* using HTTP/1.x
> GET /status/500 HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 500 Internal Server Error
< access-control-allow-credentials: true
< access-control-allow-origin: *
< content-length: 0
< content-type: text/html; charset=utf-8
< x-content-type-options: nosniff
< date: Tue, 27 Oct 2026 21:46:22 GMT
<
* Connection #0 to host httpbin.org:443 left intact
```

- En este caso, la respuesta fue “HTTP/1.1 500 Internal Server Error”.

- La responsabilidad de este problema ahora recae en el servidor, pues como se trato de acceder a un recurso que simula el propio fallo de servidor, lo genera este mismo y no podría tratarse de un error 4xx, donde el problema es del cliente.

Ejercicio 4 – Métodos distintos de GET

- Envíen un POST con cuerpo a https://httpbin.org/post (como en el ejemplo de la sección 5) y un HEAD a https://httpbin.org/get (curl -I). Anoten qué cambió en la respuesta de cada uno

- Las peticiones completas quedarían así:

curl -X POST https://httpbin.org/post
curl -I https://httpbin.org/get

```
C:\Users\JavoPC>curl -X POST https://httpbin.org/post
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Content-Length": "0",
    "Host": "httpbin.org",
    "User-Agent": "curl/8.21.0"
  },
  "method": "POST",
  "origin": "148.226.28.247",
  "url": "https://httpbin.org/post",
  "data": "",
  "files": {},
  "form": {},
  "json": null
}
```

```
C:\Users\JavoPC>curl -I https://httpbin.org/get
HTTP/1.1 200 OK
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 203
content-type: application/json
x-content-type-options: nosniff
date: Tue, 27 Oct 2026 22:13:27 GMT
```

- Ambos arrojaron información, pero el método POST regresa información basada en los parámetros que se envíen, mientras que el método HEAD solo muestra las cabeceras sin el cuerpo entero del mensaje, independientemente de quien lo pida ni cómo.

Ejercicio 5 – La petición literal con netcat

- Escriban la petición con printf + nc como en la sección 5. Comparen: ¿qué encabezados agrega curl por su cuenta que ustedes no escribieron a mano?

```
javo@JavoDeb:~$ printf "GET /get HTTP/1.1\\r\\nHost: httpbin.org\\r\\n\\r\\n"
| nc httpbin.org 80
HTTP/1.1 200 OK
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 147
content-type: application/json
x-content-type-options: nosniff
date: Tue, 27 Oct 2026 22:23:03 GMT

{
  "args": {},
  "headers": {
    "Host": "httpbin.org"
  },
  "method": "GET",
  "origin": "148.226.28.225",
  "url": "http://httpbin.org/get"
}
```

- Curl incluye otros encabezados como accept, Content-length, User-agent y algunos parámetros para información como data, files, json o form.

- Cuestionario Parte 2

Si un recurso protegido se pide sin credenciales, ¿qué código esperan: 401 o 403? ¿Cuál es la diferencia?

- Probablemente 401, pues se esperaría que tuviese las credenciales necesarias para acceder a él, mientras que 403 da a entender que bajo ninguna circunstancia debe ser accedido a ese recurso, pues no esta en espera de credenciales.

El servidor de httpbin contesta 500 cuando se lo pedimos. En un sistema real, ¿qué debería hacer el programador del servidor para que una falla no llegue al cliente como un 500 pelado?

- Para ello, debe encargarse de manejar las excepciones adecuadamente que se puedan dar a lo largo de la ejecución del código fuente de la página, evitando que algún error inesperado pueda entorpecer la comunicación y exponer algún recurso delicado.

¿Qué método y qué código usaría el servicio LA FILA si se reescribiera como API web?

- Pienso que debería utilizar el método POST para enviar información sobre los usuarios y el tipo de petición, arrojando código 200 OK cuando la comunicación se logre exitosamente, pero códigos de la suite de 4xx en caso de que tratase de acceder a un recurso inexistente o la petición estuviese mal escrita.